AeroNode

Project conversation summary | Specification, implementation, evidence, and next step

Through Phase 3: accepted multi-exit model, exported ONNX graph, and verified four-head numerical parity.

Project objective

AeroNode explores whether an edge-AI runtime can adapt inference depth to resource conditions and prediction confidence, reducing latency or compute while keeping accuracy acceptable. The binding project brief defines a staged path: Python training/export and control plane, a C++17 ONNX Runtime engine, ZeroMQ RuntimeState updates, adaptive early exits, reproducible benchmarks, then optional allocator and telemetry work.

Conversation and implementation timeline

Milestone	Outcome
Phase 0 - inspect and document	Read the full 8-page authoritative PDF, inventoried the environment, and created architecture.md, current_status.md, and README.md before runtime implementation.
Phase 1 - verified baseline	Built a deterministic PyTorch model -> ONNX -> C++17/ONNX Runtime path. Python and C++ logits matched within 1.49e-8; 500 runs measured 0.0035376 ms mean and 0.0037 ms P95 on the tiny synthetic model.
Phase 2 - multi-exit PyTorch	Trained a four-head ResNet-style classifier from scratch on the complete Imagenette 160px splits for five CPU epochs; reloaded the selected best checkpoint and independently validated every head.
Phase 3 - ONNX reference	Exported the accepted best checkpoint as a dynamic-batch, four-output ONNX graph; validated graph structure and compared all logits with ONNX Runtime on one validation image from each class.

Environment and implementation choices

The project runs on Windows 11 x64 with an Intel Core Ultra 5 225H. The available compiler is MSYS2 GCC 15.2; project-local CMake/Ninja and a Python 3.13 virtual environment are used. The Phase-1/2 stack is PyTorch 2.14.0, torchvision 0.29.0, ONNX 1.22.0, ONNX Runtime 1.30.0, and NumPy 2.5.3. The official ONNX Runtime 1.30.0 Windows x64 C++ SDK links and runs with the selected compiler. FastAPI and PostgreSQL are available but deferred; ZeroMQ is not yet installed.

The runtime plan uses staged ONNX models for future early termination because a normal ONNX Runtime session cannot promise arbitrary pause/resume inside one optimized graph. Phase 3 adds a monolithic multi-output graph as a fidelity reference. That graph exposes all exits but is not itself claimed to save compute when an early output is selected.

Next step

Phase 4: load the monolithic four-output ONNX reference in the C++ ONNX Runtime executable and compare every output with the recorded PyTorch/ONNX reference. Later work can introduce separate stage exports for actual early-exit execution; ZeroMQ and adaptive policy follow after the C++ reference path works.

Phase 2 model and validation

The prototype has four residual stages with two basic blocks per stage, widths 32/64/128/256, and approximately 2.8 million trainable parameters. Each stage feeds its own adaptive-average-pool and 10-class linear head. Weighted cross-entropy uses exit weights 0.15/0.20/0.25/0.40. Training used seed 20260916, 128x128 crops, SGD (learning rate 0.05, momentum 0.9, weight decay 0.0005), cosine decay, and batch size 64.

Exit	Validation loss	Correct / 3,925	Top-1 accuracy
1	1.378888	2,248	57.27%
2	1.053556	2,619	66.73%
3	0.875750	2,818	71.80%
4 (final)	0.829955	2,869	73.10%

Dataset: official imagenette2-160.tgz; 9,469 train / 3,925 validation images. Archive SHA-256: 64d0c4859f35a461889e0147755a999a48b49bf38a7e0f9bd27003f10db02fe5. The model was initialized randomly with no pretrained weights.

Phase 3 ONNX export and verification

The selected source was the accepted epoch-5 best_final_exit.pt checkpoint, SHA-256 32fb2975bcfe14d7311bc91db1553442391468d52c74f34a4efeb9d2aef27366. The resulting opset-17 model is 11,217,740 bytes (10.70 MiB), with SHA-256 0cc08e571383279798c00019f0079bcbc107db49a1fe8efca444a4b754f35051. It records source checkpoint, dataset, class map, loss weights, and image size as ONNX metadata.

ONNX output	Declared shape	Max abs error
exit_1_logits	[batch_size, 10]	2.3841858e-06
exit_2_logits	[batch_size, 10]	9.5367432e-07
exit_3_logits	[batch_size, 10]	1.4305115e-06
exit_4_logits	[batch_size, 10]	1.6689301e-06

Input: input, shape [batch_size, 3, 128, 128]. Numerical parity was checked for a deterministic batch of ten validation images, one per class, at absolute tolerance 1e-5 and relative tolerance 1e-4. All four outputs passed. ONNX full checker and strict shape inference passed; graph inspection found 58 nodes, 48 initializers, and four classifier paths. Netron was unavailable, so reproducible programmatic inspection was used.

Current artifacts and status

Artifact	Location
Architecture and status	C:\AeroNode\architecture.md C:\AeroNode\current_status.md
Phase 2 validation	C:\AeroNode\docs\phase2_validation.md
Phase 3 exporter and test	C:\AeroNode\model\export\export_multi_exit.py C:\AeroNode\tests\test_multi_exit_onnx.py
ONNX model and verification report	C:\AeroNode\model\artifacts\phase3\multi_exit_imagenette.onnx C:\AeroNode\model\artifacts\phase3\multi_exit_imagenette.verification.json
Phase 3 validation record	C:\AeroNode\docs\phase3_validation.md
Accepted Phase 2 checkpoint (retained)	C:\AeroNode\model\artifacts\phase2-imagenette-5ep\best_final_exit.pt

All four Phase-2 model tests and the Phase-3 ONNX test passed (4 total). The Phase-1 C++ inference baseline remains available and was previously verified. Git commits mark the Phase-1, Phase-2, and Phase-3 milestones; the project working tree was clean at the last check.